

TRAINING OF NEURAL NETWORKS

Forward Propagation, Activation Function & Backward Propagation — Notes

1. Forward Propagation

Forward propagation is the process in which input data moves through the neural network from the input layer to the output layer to produce a prediction. Each neuron takes the input, applies weights and bias, and then passes the result through an activation function. The process continues through the hidden layers until the final output is produced.

Flow:

Input → Weighted Sum → Activation Function → Hidden Layers → Output

2. Weighted Sum

Before applying an activation function, a neuron calculates a weighted sum of its inputs. Each input is multiplied by its corresponding weight, and then a bias is added.

Formula:

$$z = x_1w_1 + x_2w_2 + \dots + x_nw_n + b$$

Where:

- x = input
- w = weight
- b = bias
- z = weighted sum

3. Activation Function

An **activation function** is a mathematical function applied to the weighted sum of a neuron. It determines the output of the neuron and introduces **non-linearity** into the neural network. Non-linearity allows neural networks to learn complex patterns and relationships in data.

Flow:

Inputs → Weights + Bias → Weighted Sum → Activation Function → Output

4. ReLU Activation Function

ReLU (Rectified Linear Unit) is one of the most commonly used activation functions in hidden layers of neural networks. It returns the input if it is positive and returns zero if the input is negative.

Formula:

$$\text{ReLU}(x) = \max(0, x)$$

For example:

$$\text{ReLU}(-5) = 0$$

$$\text{ReLU}(3) = 3$$

ReLU is popular because it is simple and computationally efficient.

5. Sigmoid Activation Function

The **Sigmoid** function converts an input into a value between **0 and 1**. It is commonly used in binary classification problems where the output can represent the probability of one of two classes.

For example:

0.9 → high probability of positive class

0.2 → low probability of positive class

6. Tanh Activation Function

Tanh (Hyperbolic Tangent) converts values into a range between **-1 and +1**. It is useful in some neural networks where having both positive and negative outputs is helpful.

Range:

-1 to +1

7. Softmax Activation Function

Softmax is commonly used in the output layer for **multiclass classification**. It converts multiple output values into probabilities whose total is 1.

For example, for three classes:

Cat = 0.70

Dog = 0.20

Horse = 0.10

The model predicts the class with the highest probability, which is **Cat** in this example.

8. Loss Function

After forward propagation, the model produces a prediction. The prediction is compared with the actual answer using a **loss function**.

The loss function measures how far the prediction is from the correct result.

A **low loss** generally means the prediction is close to the expected output, while a **high loss** means the prediction is further away.

Flow:

Prediction → Compare with Actual Output → Calculate Loss

9. Backward Propagation

Backward propagation, or **backpropagation**, is the process of sending the error information backward through the neural network to determine how the model's parameters contributed to the error. It calculates gradients that show how the loss changes with respect to the model's parameters.

Flow:

Loss → Backpropagation → Gradients → Parameter Updates

Backpropagation itself calculates the gradients; an optimizer such as gradient descent uses those gradients to update the parameters.

10. Gradient

A **gradient** tells us how the loss changes when a particular parameter, such as a weight, changes. It helps determine the direction in which the parameter should be adjusted to reduce the loss.

In simple words:

Gradient tells the model how a parameter should change to reduce the error.

11. Gradient Descent

Gradient Descent is an optimization algorithm used to reduce the loss of a model. It uses the gradients calculated during backpropagation to update the model's parameters.

Basic formula:

New Weight = Old Weight - Learning Rate $\times$ Gradient

The process is repeated many times until the model learns parameters that produce better predictions.

12. Learning Rate

The **learning rate** determines how large a step the optimizer takes when updating the model's parameters.

A very large learning rate can cause the model to miss the optimal values, while a very small learning rate can make training very slow.

Therefore, a suitable learning rate is important for effective training.

13. Weights

Weights are learned values associated with the connections between neurons. They determine how strongly different inputs influence the calculations performed by the network. During training, the weights are adjusted to reduce the model's loss.

14. Bias

Bias is a learned value added to the weighted sum of a neuron. It gives the neuron additional flexibility and allows the model to shift its output when necessary.

Basic neuron calculation:

$z = \text{Weighted Sum} + \text{Bias}$

Neural Network Architecture

1. Neural Network Architecture

Neural network architecture refers to the **structure and arrangement of neurons and layers** in a neural network. It defines how input data moves through the network and how the network produces an output.

2. Basic Structure

A basic neural network consists of three main types of layers:

Input Layer → Hidden Layer(s) → Output Layer

The input layer receives data, hidden layers process the data, and the output layer produces the final prediction.

3. Input Layer

The input layer receives the **features** of the input data. Each input neuron generally represents one feature.

For example, if we want to predict house price using area, number of rooms, and location score, these can be the input features.

4. Hidden Layer

Hidden layers perform the main processing of the neural network. Each neuron calculates a weighted sum of inputs, adds a bias, and applies an activation function.

A network can have one or many hidden layers. When a neural network has multiple hidden layers, it is called a **Deep Neural Network (DNN)**.

5. Output Layer

The output layer produces the **final prediction or result** of the neural network.

The output structure depends on the problem:

- **Binary classification:** Usually one output neuron with Sigmoid activation.
- **Multiclass classification:** Usually multiple output neurons with Softmax activation.
- **Regression:** Usually one or more numerical outputs without a classification activation.

6. Neuron

A neuron is the basic processing unit of a neural network. It takes inputs, multiplies them by weights, adds a bias, and passes the result through an activation function.

Formula:

$$z = x_1w_1 + x_2w_2 + \dots + x_nw_n + b$$

Then:

$$\text{Output} = \text{Activation}(z)$$

7. Weights

Weights represent the **importance of different inputs**. During training, the neural network changes these weights to improve its predictions.

A larger or smaller weight can increase or decrease the influence of an input on the output.

8. Bias

Bias is an additional value added to the weighted sum. It helps the neuron shift its output and gives the network more flexibility when learning patterns.

9. Activation Function

An activation function is applied to the neuron's calculated value. It introduces **non-linearity**, allowing neural networks to learn complex patterns.

Common activation functions are:

- **ReLU:** Commonly used in hidden layers.
- **Sigmoid:** Commonly used for binary classification.
- **Tanh:** Produces values between -1 and +1.
- **Softmax:** Used for multiclass classification and produces probabilities.

10. Fully Connected / Dense Layer

In a fully connected layer, every neuron in one layer is connected to every neuron in the next layer.

Dense layers are commonly used in basic neural networks for classification and regression tasks.

11. Shallow and Deep Neural Networks

A **shallow neural network** has a small number of hidden layers, while a **deep neural network** contains multiple hidden layers.

Deep networks can learn more complex patterns because they perform several levels of processing.

12. Types of Neural Network Architectures

Feedforward Neural Network (FNN):

Data moves in one direction from input to output without loops. It is commonly used for basic classification and regression.

Convolutional Neural Network (CNN):

CNNs are mainly designed for image and visual data. They can identify features such as edges, shapes, and objects.

Recurrent Neural Network (RNN):

RNNs are designed for sequential data, where previous information can influence later processing. They can be used for text and time-series data.

LSTM:

Long Short-Term Memory is a type of RNN designed to handle long-term dependencies in sequential data more effectively.

Transformer:

Transformers use an attention mechanism to process relationships between different parts of the input. They are widely used in modern NLP, LLMs, and Generative AI.

13. Forward Propagation

During forward propagation, data moves from the input layer through the hidden layers to the output layer.

Input → Weighted Sum → Activation → Hidden Layers → Output

The network uses its current weights and biases to generate a prediction.

14. Backpropagation

After making a prediction, the network calculates the **loss**, which represents how incorrect the prediction is.

Backpropagation works backward through the network to calculate **gradients**, showing how the weights and biases contributed to the error. An optimizer then uses these gradients to update the parameters.

15. Neural Network Training

The neural network learns through repeated training:

Input → Forward Propagation → Prediction → Loss → Backpropagation → Weight/Bias Update → Repeat

The goal is to reduce the loss and make better predictions.

16. Neural Network Architecture Example

Suppose we have three input features:

Input Layer: 3 neurons

Then:

Hidden Layer 1: 4 neurons

Hidden Layer 2: 3 neurons

Output Layer: 1 neuron

The architecture can be represented as:

3 Inputs → 4 Hidden Neurons → 3 Hidden Neurons → 1 Output

The exact number of layers and neurons depends on the problem and is usually treated as part of the model design.